

INSTITUTO
FEDERAL
Santa Catarina

Resumo de matérias

Programação Frontend II

Eduardo Cardoso Oliveira, 2026

Sumário

1 Como estudar para esta disciplina? 6

2 Revisão de Fundamentos de JavaScript 6
2.1 Tipos de dados e variáveis . . . 6
2.2 Template literals . . . 7
2.3 Operadores . . . 7
2.4 Arrays . . . 7
2.5 Objetos . . . 8
2.6 Manipulando o DOM . . . 8

3 Paradigmas de Programação 8
3.1 O que são paradigmas . . . 8
3.2 Programação imperativa . . . 8
3.3 Programação declarativa . . . 9
3.4 Programação funcional . . . 9
3.5 Programação orientada a objetos . . . 9
3.6 JavaScript como linguagem multiparadigma . . . 9

4 Orientação a Objetos em JavaScript 10
4.1 Protótipos . . . 10
4.2 Classes . . . 10
4.3 Subclasses . . . 10
4.4 Classes abstratas . . . 11
4.5 Métodos estáticos e propriedades privadas . . . 11
4.6 Polimorfismo . . . 12

5 Programação Funcional em JavaScript (Parte 1) 12
5.1 Funções puras . . . 12
5.2 Funções de ordem superior . . . 13
5.3 Memoization . . . 13

6 Programação Funcional em JavaScript (Parte 2) 13
6.1 Recursão . . . 13
6.2 Composição de funções . . . 14
6.3 Closures . . . 14
6.4 Currying . . . 15

7 Introdução ao TypeScript 15
7.1 O que é o TypeScript . . . 15
7.2 Preparação de um projeto TypeScript . . . 15
7.3 Tipos primitivos . . . 16
7.4 Objetos e tipos personalizados . . . 16
7.5 União e interseção de tipos . . . 17
7.6 Arrays e enums . . . 17

8 Funções em TypeScript 18
8.1 Declarando funções tipadas . . . 18
8.2 Parâmetros opcionais e rest . . . 18

8.3 Definindo o tipo de uma função 18
8.4 Sobrecarga de funções 19

9 Orientação a Objetos em TypeScript 19
9.1 Generics 19
9.2 Modificadores de visibilidade 20
9.3 Classes abstratas em TypeScript 20
9.4 Interfaces 20
9.5 Tipagem estrutural 21

10 Modularização e Organização de Dependências 21
10.1 Breve histórico do uso de módulos em JavaScript 21
10.2 Módulos ES6 22
10.3 Boas práticas de modularização 22
10.4 Iniciando e configurando um projeto 23

11 Referências 23

Aviso importante e Isenção de responsabilidade

1. **Finalidade do documento**
   Este documento foi elaborado com o objetivo de servir como um material de apoio e revisão para os estudos das matérias do terceiro semestre de Análise e Desenvolvimento de Sistemas. Todos os conteúdos aqui presentes foram retirados de anotações pessoais e resumos, compilados para facilitar a compreensão do leitor.

2. **Fontes e Direitos Autorais**
   Os resumos e anotações aqui presentes foram desenvolvidos a partir dos materiais de sala disponibilizados pelos professores (como slides, apostilas, listas de exercícios, livros, etc), anotações de aula e pesquisas. Os direitos autorais sobre os materiais originais (aulas, livros, artigos) permanecem integralmente aos seus respectivos autores e editoras. Este documento é uma obra derivada, de caráter estudantil e sem fins lucrativos.

3. **Precisão das Informações**
   Desde já, o autor não oferece nenhuma garantia de que os conteúdos aqui contidos sejam precisos e corretos, por mais que todo esforço tenha sido feito para garantir que tais informações sejam corretas. Erros, omissões ou interpretações equivocadas podem existir. O autor não se responsabiliza por quaisquer consequências, acadêmicas ou de outra natureza, que possam surgir do uso deste material.

4. **Uso Recomendado**
   Este material **NÃO SUBSTITUI DE FORMA ALGUMA** a leitura de bibliografias recomendadas por professores, presença em aulas, consulta às fontes oficiais e, principalmente, as orientações do corpo docente. Utilize este documento como uma ferramenta complementar aos seus estudos.

5. **Vínculo Institucional**
   Este é um projeto independente e não possui qualquer vínculo oficial com o Instituto Federal de Santa Catarina Campus São José, a Coordenação do Curso de Análise

e Desenvolvimento de Sistemas, ou os professores das disciplinas. Não há qualquer endosso ou revisão oficial do conteúdo por parte da instituição.

Apresentação do Material

Olá, colega!

Seja bem-vindo(a) ao meu resumo de estudos! Este documento foi criado ao longo do semestre com o objetivo de centralizar e facilitar o acesso aos conteúdos que foram abordados durante as aulas da disciplina de Programação Frontend II, ministrada pelo professor Adriano Lima.

O que você vai encontrar neste documento?

- **Resumos:** Para esta disciplina trago resumo de todas as aulas, cobrindo desde uma revisão de JavaScript, passando por paradigmas de programação, orientação a objetos e programação funcional em JS, até TypeScript e modularização de projetos.
- **Exemplos de código:** Como essa disciplina é bastante prática, trago bastante trecho de código comentado, sempre baseado nos próprios exemplos usados em aula.
- **Anotações e comentários:** Anotações sobre pontos que considero mais importantes ou que costumam gerar confusão (como a diferença entre herança e composição, ou entre tipos opcionais e tipos com valor padrão no TypeScript).
- **Documentos e bibliografias:** Quando possível, vou disponibilizar materiais, listas de exercícios e referências para auxiliar.

Como este documento está organizado?

O material segue a mesma ordem das aulas do semestre: começamos com uma revisão de fundamentos de JavaScript, seguimos para os paradigmas de programação (com foco em orientação a objetos e programação funcional aplicados em JS), avançamos para o TypeScript (tipos, funções e orientação a objetos) e terminamos com modularização e organização de dependências em projetos frontend.

Informações adicionais e contato

Este material foi feito de aluno para aluno; sugestões e colaborações são sempre bem-vindas! Se encontrar algum erro, tiver alguma sugestão para adicionar ao documento ou tirar alguma dúvida sobre qualquer matéria, fique à vontade para entrar em contato.

- **Autor:** Eduardo Cardoso Oliveira
- **E-mail Institucional:** eduardo.co22@aluno.ifsc.edu.br
- **Github:** https://github.com/EduCardoso22

Sumário

1 Como estudar para esta disciplina?

Frontend II dá sequência direta ao que foi visto em Frontend I, mas aprofunda bastante o JavaScript: aqui o foco sai da manipulação de página (HTML/CSS/DOM) e vai para a lógica de programação em si, explorando diferentes paradigmas, orientação a objetos "de verdade"(com classes, herança e encapsulamento) e, principalmente, programação funcional, que tem uma forma de pensar bem diferente da programação imperativa que a gente está mais acostumado a ver. Meu conselho aqui é parecido com o de Frontend I: escreva código o tempo todo, mas com um cuidado a mais nesta disciplina, que é comparar sempre a versão "imperativa"de uma solução com a versão "funcional"dela (o professor faz isso direto nos slides), porque é exatamente esse contraste que ajuda a internalizar o novo jeito de pensar.

Outro ponto importante é notar que, embora o conteúdo de programação funcional e de TypeScript pareçam "avançados", eles seguem conceitos bem concretos e repetíveis (funções puras, imutabilidade, composição, tipos estáticos). Fazer bastante exercício prático, tentando reescrever um código imperativo simples em uma versão funcional (ou vice-versa), ajuda muito a fixar o conteúdo. Por fim, não deixe a parte de TypeScript "para depois": ela parece familiar por causa do JS, mas tem suas próprias particularidades (como tipos personalizados, genéricos e modificadores de visibilidade) que exigem prática própria.

2 Revisão de Fundamentos de JavaScript

Como o nome sugere, essa primeira aula retoma e consolida os principais conceitos de JavaScript que são pré-requisito para o resto da disciplina (muitos deles já vistos em Frontend I), então vou tratar essa seção como uma referência rápida, com foco nos detalhes que considero mais importantes de revisar.

2.1 Tipos de dados e variáveis

Os tipos primitivos principais do JavaScript são: `undefined` (indefinido), `null` (nulo), `number` (número), `string` e `boolean` (booleano). Entre os valores booleanos, vale lembrar quais valores são considerados "falsy"(tratados como falso em um contexto booleano): `null`, `undefined`, `0` e a string vazia ; todo o resto é considerado "truthy".

Quanto à declaração de variáveis, a diferença central entre `var`, `let` e `const` está no escopo: `var` tem escopo de função, enquanto `let` e `const` têm escopo de bloco (respeitam chaves {}, como dentro de um `if`); além disso, `const` não pode ser reatribuída depois de declarada:

```
let x = 1;
if (x === 1) {
  let x = 2;
  console.log(x); // Imprime 2 (escopo de bloco)
}
console.log(x); // Imprime 1
let x = 3; // ERRO! x ja foi declarada neste escopo
```

2.2 Template literals

Os **template literals** são strings delimitadas por crase (‘) que permitem interpolar expressões JavaScript diretamente dentro do texto, usando a sintaxe `${ expressao }`. Podem abranger múltiplas linhas e usar os mesmos caracteres de escape de uma string comum:

```
let nome = "Mario";
let saudacao = ‘Ola, ${nome}!‘;
console.log(saudacao); // Ola, Mario!
```

2.3 Operadores

Entre os operadores relacionais, é fundamental não confundir `==` (compara apenas o valor, convertendo tipos se necessário) com `===` (compara valor e tipo, sem nenhuma conversão): `"5"== 5` resulta em `true`, mas `"5"=== 5` resulta em `false`. Os operadores lógicos seguem a tabela-verdade clássica:

x	y	!x	x —— y	x && y
true	true	false	true	true
true	false	false	true	false
false	true	true	true	false
false	false	true	false	false

2.4 Arrays

Arrays em JavaScript são grupos de valores ordenados, dinâmicos (podem crescer e diminuir) e podem ser ”esparsos”(elementos sem índices contíguos, como em `[1, , 3]`). O **operador de propagação** (*spread*, `...`) insere os elementos de um array dentro de outro array, e funciona com qualquer objeto iterável:

```
let x = [1, 2, 3];
let y = [0, ...x, 4]; // y == [0, 1, 2, 3, 4]
let algarismos = [..."0123456789"];
// algarismos == ["0","1","2",...,"9"]
```

Entre os métodos de array mais usados estão `push()`/`pop()` (adicionam/removem do final), `unshift()`/`shift()` (adicionam/removem do início), `forEach()`, `map()`, `filter()`, `find()`/`findIndex()` (como o `filter()`, mas param no primeiro elemento que satisfaz a condição), `concat()` (concatena arrays), `slice()` (retorna um subconjunto, sem alterar o array original) e `splice()` (insere e/ou remove elementos diretamente no array original):

```
let m = [1,2,3,4,5,6,7,8];
m.splice(4); // m vira [1,2,3,4]; retorna [5,6,7,8]
m.splice(1,2); // m vira [1,4]; retorna [2,3]

let n = [1,2,3,4,5];
n.splice(2,0,"a","b"); // n vira [1,2,"a","b",3,4,5]
```

2.5 Objetos

Um objeto é uma coleção não ordenada de propriedades, cada uma com um nome (geralmente uma string) e um valor (primitivo ou outro objeto). As propriedades são acessadas e escritas pelos operadores ponto (`.`) e colchetes (`[]`), podem ser acessadas condicionalmente com o **encadeamento opcional** (`?.` e `?.[]`, que retorna `undefined` em vez de gerar um erro quando uma propriedade intermediária não existe) e podem ser removidas com `delete`:

```
let local = livro.autor.nascimento?.local; // undefined, sem erro
delete livro.edicao;
```

A **serialização** converte o estado de um objeto em uma string da qual ele pode ser restaurado depois, usando o formato **JSON**: `JSON.stringify()` serializa o objeto, e `JSON.parse()` o restaura.

2.6 Manipulando o DOM

Para selecionar elementos, além de `getElementById()` e `getElementsByTagName()`, os métodos mais modernos e flexíveis são `querySelector()` e `querySelectorAll()`, que aceitam qualquer seletor CSS. Para atributos, usa-se `getAttribute()`, `setAttribute()`, `hasAttribute()` e `removeAttribute()`; para classes, a propriedade `classList` oferece `add()`, `remove()`, `toggle()` e `contains()`.

Para criar, inserir e remover elementos dinamicamente: `createElement()` cria um novo elemento; `append()` e `prepend()` inserem no final ou no início de um elemento pai; `after()` e `before()` inserem em relação a um elemento irmão; e `remove()` e `replaceWith()` removem ou substituem um elemento existente:

```
const item = document.createElement('li');
item.textContent = 'Bife acebolado';
const cardapio = document.querySelector('ul');
cardapio.append(item); // insere no final
cardapio.children[2].remove(); // remove o filho da posicao 2
```

3 Paradigmas de Programação

3.1 O que são paradigmas

Um **paradigma de programação** é um estilo ou abordagem de programação: um conjunto de conceitos, técnicas e princípios que guiam o desenvolvimento de software e definem como os problemas são resolvidos. É importante entender que uma mesma linguagem pode suportar mais de um paradigma (como o próprio JavaScript, que é multiparadigma).

3.2 Programação imperativa

É o paradigma mais antigo, semelhante ao padrão imperativo das línguas naturais (uma sequência de comandos, como uma receita de bolo): o foco está em **como** o computador deve executar as instruções, que são executadas na ordem em que aparecem. Suas características principais são: estado do sistema (variáveis que mudam ao longo

da execução), instruções sequenciais, controle de fluxo explícito (`if`, `for`), variáveis e atribuições, efeitos colaterais, abstração procedural e mutabilidade:

```
let contador = 0;
function incrementarContador() {
  contador++;
  console.log(`O contador agora e: ${contador}`);
}
incrementarContador();
incrementarContador();
```

3.3 Programação declarativa

Nesse paradigma, o foco está em **o que** deve ser feito, e não em como fazer. Tem sintaxe mais intuitiva e permite a definição de restrições e regras, sendo bastante usado em inteligência artificial e em acesso a bancos de dados (como o próprio SQL). Suas características incluem: declaração de intenção, imutabilidade, funções puras, ausência de controle de fluxo explícito, recursão, abstração de estado e uso de expressões em vez de instruções:

```
const numeros = [1, 2, 3, 4, 5, 6];
let pares = numeros.filter(num => num % 2 === 0);
let soma = pares.reduce((acc, num) => acc + num, 0);
console.log(pares); // [2, 4, 6]
console.log(soma); // 12
```

3.4 Programação funcional

A programação funcional é um subtipo de programação declarativa, baseada na avaliação de funções matemáticas, evitando o uso de estado e de mutabilidade. A maioria das linguagens modernas (incluindo o JavaScript) incorpora aspectos funcionais. Suas características incluem funções de primeira classe, imutabilidade, funções puras, transparência referencial, recursão, composição de funções, avaliação preguiçosa e uso de coleções imutáveis.

3.5 Programação orientada a objetos

É uma abordagem centrada em objetos para modelar o mundo real, em que objetos são instâncias de classes e possuem atributos (propriedades) e comportamento (métodos), resultando em código mais modular, reutilizável e fácil de entender. Suas características principais são: objetos e classes, encapsulamento, herança, polimorfismo, abstração, reusabilidade e composição.

3.6 JavaScript como linguagem multiparadigma

Na prática, o JavaScript combina os três paradigmas conforme a necessidade: o paradigma **imperativo** é usado para tarefas simples ou de controle de fluxo; o **funcional**, para manipulação de dados e cálculos matemáticos; e o **orientado a objetos**, para modelagem de entidades do mundo real e interfaces de usuário complexas.

4 Orientação a Objetos em JavaScript

4.1 Protótipos

Diferente de linguagens como Java, o JavaScript é baseado em **protótipos**, e não em classes "puras": quase todo objeto JavaScript tem um protótipo associado, do qual herda suas propriedades. Objetos literais herdam de `Object.prototype`, enquanto objetos criados com um construtor herdam do protótipo daquele construtor. É possível criar um objeto definindo explicitamente seu protótipo com `Object.create()`:

```
let o = {};
o.x = 1;
let p = Object.create(o);
p.y = 2;
let q = Object.create(p);
q.z = 3;
q.x + q.y; // 3 (herdados via cadeia de prototipos)
```

A herança em JavaScript é **dinâmica**: propriedades não encontradas em um objeto são buscadas no seu protótipo (e no protótipo do protótipo, formando a "cadeia de protótipos"), e as propriedades são compartilhadas, não copiadas. Propriedades herdadas podem ser sobrescritas em um objeto filho sem alterar o protótipo original.

4.2 Classes

Uma classe é, essencialmente, um conjunto de objetos que compartilham o mesmo protótipo. Antes do ES6, classes eram simuladas com funções construtoras e manipulação direta do `prototype`; a partir do ES6, a sintaxe `class` formaliza isso de maneira mais clara (embora continue usando protótipos por trás dos panos):

```
class Retangulo {
  constructor(largura, altura) {
    this.largura = largura;
    this.altura = altura;
  }
  area() {
    return this.largura * this.altura;
  }
  perimetro() {
    return 2 * (this.largura + this.altura);
  }
}
let ret = new Retangulo(2, 5);
```

4.3 Subclasses

Subclasses herdam as propriedades e os métodos de suas superclasses, podendo definir métodos próprios e sobrescrever os métodos herdados. Com a sintaxe de classes do ES6, isso é feito com `extends`, e o construtor da subclasse precisa invocar o construtor da superclasse com `super()` antes de poder usar `this`:

```
class Quadrado extends Retangulo {
  constructor(lado) {
    super(lado, lado);
  }
}
```

Uma alternativa à herança é a **composição** (ou delegação): em vez de uma classe herdar de outra, ela mantém uma instância da outra classe internamente e delega o comportamento para ela. É importante destacar que isso **não** é herança: é uma forma diferente de reaproveitar código, que costuma gerar um acoplamento mais flexível entre as classes.

4.4 Classes abstratas

Uma classe abstrata é uma classe que não pode ser instanciada diretamente, servindo apenas como base para subclasses. Ela pode conter métodos concretos (já implementados, que podem ser reutilizados ou sobrescritos) e métodos abstratos (que devem obrigatoriamente ser implementados pelas subclasses). É importante lembrar que classes abstratas **não existem formalmente em JavaScript**: o professor simula esse comportamento lançando um erro dentro do método que deveria ser abstrato, caso ele não tenha sido sobrescrito pela subclasse:

```
class Figura {
  area() {
    throw new Error('Metodo nao implementado');
  }
}
class Retangulo extends Figura {
  constructor(largura, altura) {
    super();
    this.largura = largura;
    this.altura = altura;
  }
  area() {
    return this.largura * this.altura;
  }
}
```

4.5 Métodos estáticos e propriedades privadas

Métodos estáticos (definidos com `static`) estão associados diretamente à classe, e não às suas instâncias; são usados principalmente para métodos utilitários ou operações globais. Já as **propriedades privadas** (marcadas com o prefixo `#`) só podem ser acessadas internamente, dentro da própria classe, e precisam ser declaradas antes de usadas:

```
class Retangulo {
  #largura;
  #altura;
  constructor(largura, altura) {
    this.#largura = largura;
```

```
    this.#altura = altura;
  }
  get largura() {
    return `${this.#largura} cm`;
  }
  set largura(valor) {
    if (typeof valor === 'string') valor = parseInt(valor);
    this.#largura = valor;
  }
}
```

Os **getters** e **setters** permitem acessar e modificar propriedades de forma controlada, enquanto, do ponto de vista de quem usa a classe, parecem ser acessadas diretamente (sem precisar chamar explicitamente um método), permitindo encapsular o estado interno da classe.

4.6 Polimorfismo

Polimorfismo significa ter o mesmo nome de método com diferentes "assinaturas"(comportamentos diferentes dependendo dos parâmetros recebidos). O JavaScript **não suporta polimorfismo nativamente**, então ele costuma ser simulado verificando a quantidade ou o tipo dos parâmetros recebidos dentro da própria função:

```
function calcularArea(...args) {
  if (args.length === 1) {
    return Math.PI * args[0] ** 2; // area do circulo
  } else if (args.length === 2) {
    return args[0] * args[1]; // area do retangulo
  }
}
```

5 Programação Funcional em JavaScript (Parte 1)

5.1 Funções puras

Uma função pura tem duas características principais: **determinismo** (as mesmas entradas sempre produzem os mesmos resultados) e **ausência de efeitos colaterais** (não altera nenhum estado fora do seu próprio escopo). É importante perceber que algumas funções nativas do array são puras por padrão, e outras não: o método `sort()` é sempre impuro, pois modifica o array original; já o `map()` é puro, desde que não dependa de nenhuma variável externa dentro da função passada a ele:

```
const numeros = [4, 3, 2, 1];

// sort() e sempre impuro, modifica o array original
const ordenados = numeros.sort((a, b) => a - b);

// map() e puro aqui, nao depende de nada externo
const dobro = numeros.map(n => n * 2);
```

```
// mas este map() e impuro, depende de uma variavel externa
let multiplicador = 2;
const dobroImpuro = numeros.map(n => n * multiplicador);
```

5.2 Funções de ordem superior

Uma **função de ordem superior** é aquela que opera sobre outras funções: recebe funções como argumento e/ou retorna uma função como resultado. Isso é possível porque, em JavaScript, funções são "cidadãs de primeira classe"(podem ser tratadas como qualquer outro valor: armazenadas em variáveis, passadas como parâmetro, retornadas de outras funções):

```
const somar = a => b => a + b;
const somarCinco = somar(5); // somarCinco = b => 5 + b
console.log(somarCinco(10)); // 15
```

5.3 Memoization

Memoization é uma técnica de otimização que guarda em "memória"(em um cache) os resultados de chamadas anteriores de uma função, evitando recalcular o mesmo resultado várias vezes. Melhora bastante o desempenho de algoritmos recursivos, mas tem como desvantagem o consumo adicional de memória:

```
function fatorialMemo() {
  let cache = {};
  return function fatorial(n) {
    if (n in cache) return cache[n];
    if (n === 0 || n === 1) return 1;
    cache[n] = n * fatorial(n - 1);
    return cache[n];
  };
}
const fatorial = fatorialMemo();
console.log(fatorial(5)); // 120 (calcula)
console.log(fatorial(5)); // 120 (vem do cache)
```

6 Programação Funcional em JavaScript (Parte 2)

6.1 Recursão

Uma função recursiva é aquela que invoca a si mesma repetidamente até atingir uma condição de parada, podendo ser dividida em duas partes: o **caso base** (a condição que interrompe a recursão) e o **passo indutivo** (a chamada da função sobre um problema menor). É equivalente, em certo sentido, à indução matemática, e é especialmente adequada para estruturas naturalmente recursivas, como árvores (incluindo o DOM), menus aninhados ou JSON aninhado:

```
// funcao recursiva para percorrer todos os elementos do DOM
function percorrerDOM(elemento) {
```

```
  if (elemento.tagName) {
    console.log(elemento.tagName);
  }
  let filhos = Array.from(elemento.children);
  filhos.forEach(filho => { percorrerDOM(filho); });
}
```

As principais desvantagens da recursão são o custo de desempenho (comparado a um laço equivalente), a maior dificuldade de entendimento para quem lê o código, e o risco de causar um **estouro de pilha** (*stack overflow*) em recursões muito profundas.

6.2 Composição de funções

A **composição de funções** consiste em construir funções mais complexas a partir de funções simples, de forma que o resultado de uma função seja passado diretamente como entrada para a próxima ($f(g(x))$). A principal vantagem é a modularidade: cada função realiza uma única operação, o que as torna mais fáceis de reutilizar, manter e modificar. Pode ser feita manualmente, por encadeamento de métodos (como `filter`, `map` e `reduce` em sequência), ou através de uma função utilitária de **pipe**, que aplica uma lista de funções em sequência sobre um valor inicial:

```
const ePar = n => n % 2 === 0;
const quadrado = n => n ** 2;
const soma = (a, b) => a + b;

const pipe = (...funcoes) => {
  return x => funcoes.reduce((v, f) => f(v), x);
};

const somaParesAoQuadrado = pipe(
  arr => arr.filter(ePar),
  arr => arr.map(quadrado),
  arr => arr.reduce(soma, 0)
);

somaParesAoQuadrado([1,2,3,4,5,6]); // 56
```

6.3 Closures

Uma **closure** é o conjunto formado por uma função e as variáveis que ela ”capturou”do escopo léxico em que foi criada: a função ”lembra”do ambiente onde nasceu, mesmo depois que esse ambiente, em princípio, já teria deixado de existir. Isso ocorre quando uma função é criada dentro de outra função, e a função interna usa variáveis da função externa:

```
function criaContador() {
  let contador = 0;
  return function() {
    contador++;
    console.log(contador);
  };
}
```

```
let contadorClique = criaContador();
const botao = document.getElementById("botao");
botao.addEventListener("click", contadorClique);
```

As closures protegem dados e detalhes da implementação (funcionando como uma forma de encapsulamento) e mantêm o estado de uma função sem depender de variáveis globais ou de objetos externos. A técnica de memoization, vista anteriormente, é na verdade um exemplo prático de closure: a variável `cache` fica ”presa”dentro da função retornada.

6.4 Currying

Currying é a transformação de uma função com vários parâmetros em uma sequência de funções, cada uma recebendo apenas um parâmetro. Isso permite ”fornecer os argumentos aos poucos”, criando funções cada vez mais específicas a partir de uma função mais genérica:

```
// funcao convencional
function multiplicar(a, b) {
  return a * b;
}

// funcao com currying
function multiplicar(a) {
  return function(b) {
    return a * b;
  };
}

multiplicar(3)(10); // 30
const triplica = multiplicar(3);
triplica(10); // 30
```

A principal vantagem do currying é economizar esforço na escrita de funções especializadas (como `triplica` acima, derivada de `multiplicar`); a desvantagem é que pode tornar o código visualmente mais complexo e um pouco mais difícil de ler para quem não está acostumado com o padrão.

7 Introdução ao TypeScript

7.1 O que é o TypeScript

O **TypeScript** é um superconjunto do JavaScript que adiciona tipagem estática à linguagem, o que permite detectar erros precocemente (antes mesmo de executar o código), melhora a legibilidade e a manutenção, contribui para a escalabilidade de projetos maiores e aprimora bastante a ajuda oferecida por IDEs (autocompletar, refatoração, etc).

7.2 Preparação de um projeto TypeScript

O TypeScript exige Node.js com npm instalado. Pode ser instalado globalmente (`npm install -g typescript`) ou localmente, como dependência de desenvolvimento do

projeto (`npm install -D typescript`). A configuração do compilador fica no arquivo `tsconfig.json`, que pode ser criado manualmente ou gerado automaticamente com `npx tsc --init`:

```
{
  "compilerOptions": {
    "target": "ES2016",
    "outDir": "./dist/js",
    "noEmitOnError": true,
    "module": "commonjs",
    "moduleResolution": "node",
    "strict": true
  },
  "include": ["./src/**/*.ts"]
}
```

A compilação pode ser feita arquivo a arquivo (`tsc arquivo.ts`) ou de forma automática, recompilando a cada alteração salva (`tsc -w`, de *watch*).

7.3 Tipos primitivos

Um **tipo**, em TypeScript, é um conjunto de valores e o que pode ser feito com eles. Os tipos podem ser definidos explicitamente (`let a: number = 1`) ou inferidos automaticamente pelo compilador a partir do valor atribuído (`let a = 1`). Além dos tipos básicos (`number`, `string`, `boolean`), existem dois tipos especiais:

- `any`: desliga a verificação de tipos para aquela variável, permitindo qualquer operação sobre ela (inclusive operações inválidas, que só seriam percebidas em tempo de execução). Deve ser evitado sempre que possível, pois anula a principal vantagem do TypeScript.
- `unknown`: também aceita qualquer valor, mas, diferente do `any`, exige uma verificação de tipo (como um `typeof`) antes de permitir operações sobre o valor, sendo uma alternativa bem mais segura ao `any`.

```
let a: unknown = 30;
let b = a === 123; // ok, comparacao e sempre permitida
let c = a + 10; // Erro TS2571: Object is of type 'unknown'
if (typeof a === 'number') {
  let d = a + 10; // ok, dentro do if o TS sabe que 'a' e number
}
```

7.4 Objetos e tipos personalizados

O formato de um objeto pode ser definido explicitamente, indicando quais propriedades são obrigatórias, opcionais (com `?`) ou de quantidade indefinida (com uma *index signature*):

```
let obj: {
  prop: number; // obrigatoria
  nome?: string; // opcional
```

```
  [chave: number]: boolean; // quantidade qualquer
} = { prop: 12, nome: 'Joao', 1: true, 2: false };
```

Os **tipos personalizados** (*type aliases*, definidos com `type`) funcionam como "apelidos"para tipos, ajudando no entendimento do código. Diferente dos tipos primitivos, eles não são inferidos: precisam ser definidos explicitamente, e têm escopo de bloco:

```
type Matricula = number;
type Pessoa = {
  matricula: Matricula;
  nome: string;
  funcao: string;
};
let novoAluno: Pessoa = {
  matricula: 202421232,
  nome: 'Joao',
  funcao: 'Aluno'
};
```

7.5 União e interseção de tipos

O operador `|` (**união**) permite que uma variável aceite valores de mais de um tipo diferente; o operador `&` (**interseção**) combina múltiplos tipos em um único tipo, exigindo que o valor satisfaça todos eles ao mesmo tempo:

```
let id: string | number;
id = "ABC123"; // valido
id = 456; // tambem valido

type Pessoa = { nome: string; idade: number };
type Funcionario = { id: number; departamento: string };
type PessoaFuncionario = Pessoa & Funcionario;

const funcionario: PessoaFuncionario = {
  nome: "Alice", idade: 30, id: 123, departamento: "Engenharia"
};
```

7.6 Arrays e enums

Arrays em TypeScript podem ser declarados com tipo inferido, anotados explicitamente (`let numeros: number[]` ou `let numeros: Array<number>`) ou marcados como somente leitura com `readonly`, o que impede métodos que alterem o array, como `push()`.

Já os **enums** (enumeráveis) representam um conjunto de constantes nomeadas, cujos valores podem ser do tipo `number` (incrementados automaticamente a partir de 0, a menos que especificado) ou `string`:

```
enum TipoFuncao {
  ALUNO = 'Aluno',
  PROFESSOR = 'Professor',
  FUNCIONARIO = 'Funcionario'
```

```
}
type Pessoa = {
  matricula: Matricula;
  nome: string;
  funcao: TipoFuncao;
};
```

8 Funções em TypeScript

8.1 Declarando funções tipadas

Funções em TypeScript podem ser declaradas de todas as formas já conhecidas do JavaScript (nomeada, expressão de função, função seta), mas com a adição de tipos para os parâmetros e para o retorno:

```
function saudacao(nome: string): string {
  return 'ola ' + nome;
}
let saudacaoSeta = (nome: string): string => 'ola ' + nome;
```

8.2 Parâmetros opcionais e rest

Parâmetros **opcionais** são marcados com `?` e devem obrigatoriamente vir depois dos parâmetros obrigatórios na assinatura da função. Também é possível dar um **valor padrão** a um parâmetro (sem usar `?` nesse caso), que é usado quando o argumento não é informado na chamada:

```
function log(message: string, userId = "Nao conectado") {
  let time = new Date().toLocaleTimeString();
  console.log(time, message, userId);
}
log("Pagina carregada");
// Logs "12:38:31 PM Pagina carregada Nao conectado"
```

Os **parâmetros rest** (`...args`) permitem que uma função aceite qualquer número de argumentos (funções variádicas), sendo úteis para implementar uma forma de polimorfismo de funções:

```
function soma(...numbers: number[]): number {
  return numbers.reduce((total, n) => total + n, 0);
}
soma(1, 2, 3); // 6
```

8.3 Definindo o tipo de uma função

É possível definir o tipo de uma função usando `type`, especificando a assinatura esperada (parâmetros e retorno), o que é útil quando várias funções precisam seguir o mesmo "contrato":

```
type Operacao = (a: number, b: number) => number;
const soma: Operacao = (a, b) => a + b;
const subtracao: Operacao = (a, b) => a - b;
```

8.4 Sobrecarga de funções

A **sobrecarga** (*overload*) permite definir diferentes assinaturas possíveis para a mesma função, é composta por uma ou mais assinaturas "sem corpo"(apenas declarando os tipos aceitos) seguidas por uma única implementação concreta, compatível com todas as assinaturas declaradas:

```
function formatar(data: Date): string;
function formatar(texto: string, maiuscula: boolean): string;
function formatar(a: Date | string, b?: boolean): string {
  if (a instanceof Date) {
    return a.toISOString();
  }
  if (typeof a === "string") {
    return b ? a.toUpperCase() : a.toLowerCase();
  }
  return "";
}
console.log(formatar(new Date()));
console.log(formatar("Ola Mundo", true)); // OLA MUNDO
```

9 Orientação a Objetos em TypeScript

9.1 Generics

Um **generic** é um parâmetro de tipo polimórfico, usado quando o tipo exato de um valor ainda é desconhecido no momento em que a função ou a classe é escrita, mas precisa ser mantido consistente em vários pontos daquele código. O tipo genérico é indicado pela convenção `<T>`:

```
type Filter = <T>(
  array: T[],
  f: (item: T) => boolean
) => T[];

let filter: Filter = (array, f) => {
  let result = [];
  for (let i = 0; i < array.length; i++) {
    if (f(array[i])) result.push(array[i]);
  }
  return result;
};
```

9.2 Modificadores de visibilidade

Assim como em outras linguagens orientadas a objetos ”clássicas”, o TypeScript suporta modificadores de visibilidade para propriedades e métodos de uma classe: `public` (padrão, acessível de qualquer lugar), `private` (acessível apenas dentro da própria classe) e `protected` (acessível na própria classe e em suas subclasses). Uma forma bastante concisa de declarar propriedades privadas é direto na assinatura do construtor:

```
class Retangulo {
  constructor(
    private _largura: number,
    private _altura: number
  ) {}
  protected area(): number {
    return this._largura * this._altura;
  }
  public perimetro(): number {
    return 2 * (this._largura + this._altura);
  }
}
```

9.3 Classes abstratas em TypeScript

Diferente do JavaScript puro, o TypeScript suporta classes abstratas **nativamente**, com a palavra-chave `abstract`. Uma classe abstrata não pode ser instanciada diretamente, e métodos marcados como `abstract` devem obrigatoriamente ser implementados por qualquer subclasse concreta:

```
abstract class Transporte {
  constructor(public nome: string) {}
  abstract mover(): void;
  parar(): void {
    console.log(‘${this.nome} parou.‘);
  }
}
class Carro extends Transporte {
  mover(): void {
    console.log(‘${this.nome} esta dirigindo.‘);
  }
}
```

9.4 Interfaces

Uma **interface** define um ”contrato”que uma classe deve seguir: quais propriedades e métodos ela precisa ter, sem dizer como eles devem ser implementados. Uma classe pode implementar mais de uma interface ao mesmo tempo, com `implements`, e interfaces também podem ser genéricas:

```
interface Animal {
  readonly nome: string;
  comer(comida: string): void;
```

```
}
interface Felino {
  miar(): void;
}
class Gato implements Animal, Felino {
  nome: string = 'Bichano';
  comer(comida: string) { console.log('Comi', comida); }
  miar() { console.log('Miau!'); }
}
```

9.5 Tipagem estrutural

Uma característica importante do TypeScript é que suas classes são **estruturalmente tipadas**: duas classes com a mesma "forma"(mesmas propriedades e métodos) são consideradas compatíveis entre si, mesmo sem nenhuma relação de herança explícita entre elas. A exceção são classes com campos `private` ou `protected`, que quebram essa compatibilidade estrutural:

```
class Zebra {
  mover() { console.log("A zebra esta trotando!"); }
}
function passear(animal: Zebra) {
  animal.mover();
}
class Poodle {
  mover() { console.log("O poodle esta andando!"); }
}
let poodle = new Poodle();
passear(poodle); // funciona! Poodle tem a mesma "forma" que Zebra
```

10 Modularização e Organização de Dependências

10.1 Breve histórico do uso de módulos em JavaScript

Entender a evolução histórica da modularização em JavaScript ajuda a entender por que ela é tão importante hoje. Nos anos 90 até o início dos anos 2000, o JS era visto apenas como um complemento ao HTML e ao CSS, com trechos de código embutidos diretamente nas páginas, o que gerava sérios problemas de manutenção, conflitos entre variáveis e funções globais, e quase nenhum reaproveitamento de código. Nos anos 2000, passou-se a usar arquivos JS externos (incluídos com `<script src="...>`), reduzindo a poluição visual, mas ainda concentrando tudo em um único arquivo gigante conforme o projeto crescia.

Entre 2005 e 2010, com aplicações mais dinâmicas (a chamada "Web 2.0") e bibliotecas como jQuery, surgiram técnicas como as **IIFE** (funções imediatamente invocadas, que criam um escopo próprio para evitar poluir o escopo global) e o uso de **namespaces** (agrupar tudo dentro de um único objeto global), mas cada biblioteca tinha seu próprio jeito de "modularizar", sem um padrão oficial para o ecossistema.

Entre 2010 e 2015, surgiram padrões mais formais: o **CommonJS** (usado no Node.js, com `require` e `module.exports`, voltado principalmente para o backend) e o **AMD**

(Asynchronous Module Definition, com `define` e `require`, voltado para o frontend), além de *bundlers* como Browserify e Webpack. O problema era ter dois padrões diferentes competindo entre si, com uma sintaxe relativamente complexa para rodar diretamente no navegador.

Finalmente, a partir de 2015, os módulos foram padronizados oficialmente com o **ES6**, trazendo as palavras-chave `import` e `export`. Essa padronização trouxe benefícios importantes: um padrão oficial, com melhor legibilidade e manutenção, funcionando tanto no navegador quanto no Node.js, e se tornou a base para os frameworks modernos e para a integração com TypeScript e bundlers.

10.2 Módulos ES6

Com os módulos ES6, cada arquivo passa a ser um módulo independente: variáveis, funções e classes são privadas ao módulo por padrão, e precisam ser explicitamente exportadas (com `export`) para serem usadas em outro arquivo, e explicitamente importadas (com `import`) por quem for utilizá-las. No navegador, um módulo é adicionado à página com `<script type="module>`.

Existem duas formas de exportação: a **nomeada** (usando apenas `export`, permite exportar vários elementos de um mesmo arquivo, cada um com seu próprio nome) e a **default** (usando `export default`, limitada a apenas um elemento "principal"por arquivo). É possível combinar as duas no mesmo arquivo (uma exportação default mais uma ou mais exportações nomeadas):

```
// exportacao nomeada
export function soma(a, b) { return a + b; }
export function subtrai(a, b) { return a - b; }

// exportacao default
export default class Usuario {
  constructor(nome) {
    this.nome = nome;
  }
}
```

A importação segue o mesmo padrão: importação nomeada (com chaves { }, podendo usar apelidos com `as`, ou importar tudo de um módulo com `* as`) e importação default (sem chaves, podendo usar qualquer nome):

```
// importacao nomeada
import { soma, subtrai } from './utils/math.js';
import { upper as paraMaiusculo } from './utils/string.js';
import * as constantes from './utils/constantes.js';

// importacao default
import multiplica from './utils/multiplica.js';
```

10.3 Boas práticas de modularização

O professor destaca algumas boas práticas para organizar um projeto frontend de forma modular: cada módulo deve ter apenas uma responsabilidade; organizar as pastas por

domínio (como `components/`, `pages/`, `services/`, `utils/`); usar nomes claros, evitando nomes genéricos como `index2.js` ou `teste.js`; agrupar funções relacionadas no mesmo módulo; e reutilizar código seguindo o princípio DRY (*Don't Repeat Yourself*), criando bibliotecas internas de funções comuns.

10.4 Iniciando e configurando um projeto

Para iniciar um projeto frontend moderno, o pré-requisito é ter o Node.js com npm instalado (ou alternativas como Yarn ou pnpm). O fluxo básico é criar o diretório do projeto e inicializar o `package.json` (de forma interativa, com `npm init`, ou automática, com `npm init -y`), seguido da instalação das dependências, que podem ser **de execução** (`npm install --save axios`) ou **de desenvolvimento** (`npm install -D eslint`).

O arquivo `package.json` identifica o projeto, gerencia suas dependências, padroniza os scripts disponíveis e garante a reprodutibilidade do ambiente (quando o projeto é clonado por outra pessoa, basta rodar `npm install` para reinstalar todas as dependências necessárias):

```
{
  "name": "meu-projeto-frontend",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "jest"
  },
  "dependencies": {
    "react": "^18.0.0",
    "axios": "^1.7.0"
  },
  "devDependencies": {
    "vite": "^5.0.0"
  }
}
```

11 Referências

Referências

[1] GOOGLE DRIVE. Programação Frontend II. Disponível em: `https://drive.google.com/drive/folders/1dEb0_tHw1J4JrIB_HCDTadKseohhAACM?usp=drive_link`. Acesso em: 30 out. 2026.

[2] LIMA, Adriano. Material de aula — Programação Frontend II. Instituto Federal de Santa Catarina, Campus São José, 2025.